Chapter 1: Introduction

Object Space vs. Image Space

Space	Description
Object Space	Continuous abstract space where shapes are defined using coordinates
Image Space	Discrete pixel grid (display monitor) where the image is produced

Key Areas

- **Geometric Representation** — defining objects using coordinates (vertices, etc.)
- **Transformation** — mapping objects to image space using matrices (controls position, orientation, size)
- **Scan Conversion** — converting continuous figures to discrete pixels (rasterization)
- **Anti-aliasing** — reducing the "staircase" distortion caused by scan conversion

2D Graphics Pipeline

$$\text{Object Coordinates} \rightarrow \text{World Coordinates} \rightarrow \text{Device Coordinates}$$

3D Graphics Concepts

- **Projection** — mapping 3D objects onto a 2D display surface
- **Hidden Surface Removal** — hiding surfaces obscured from the viewer
- **Illumination & Shading** — simulating light reflection for realism
  - *Local illumination* — direct light only
  - *Global illumination* — includes reflected light and transparency

Allied Fields

- **Image Processing** — pixel-based operations on existing images (not synthesizing from scratch)
- **Computer-Human Interaction** — interfaces and logical input devices

---

Chapter 2: Image Representation

Digital Image Basics

Term	Definition
Resolution	Number of pixels per unit length (e.g., pixels per inch)
Aspect Ratio	Width ÷ Height (e.g., 1920 × 1080 → 16:9)

Color Models

RGB (Additive — for displays)

$$\text{Color} = (R, G, B), \quad R, G, B \in [0, 1]$$

- Black = (0,0,0), White = (1,1,1)
- The **gray axis** is the diagonal from black to white

CMY (Subtractive — for printers)

$$C = 1-R, \quad M = 1-G, \quad Y = 1-B$$

Image Representation Methods

Direct Coding

Each pixel stores actual color values.

Type	Bits/pixel	Colors
Binary (B&W)	1	2
Grayscale	8	256
True color (RGB)	24	~16.7 million

Lookup Table (Color Map)

Each pixel stores an **index** into a color table.

- 8-bit pixel → 256-entry table, each entry = 24-bit RGB
- **Memory formula:** `Entries × 3 bytes`
  - Example: 2-byte pixel → 65,536 entries → 65,536 × 3 = **196,608 bytes**

Output Devices

CRT Monitors

- Electron guns strike phosphor-coated screen → emits light
- Requires refresh (~60 Hz) to avoid flicker
- Uses 3 guns (R, G, B) with shadow masks

Printers (CMYK)

- Paper is white; ink **subtracts** light
- Black ink used separately (cheaper, better quality than mixing C+M+Y)

Techniques to simulate shades:

Technique	Method	Resolution
Halftoning	Varies dot size in a grid	Reduced
Dithering	Threshold matrix decides pixel ON/OFF	Preserved
Error Diffusion	Spreads rounding error to neighbors	Preserved

Dithering rule: Pixel ON if `ImageIntensity(x,y) > D_n(i,j)`, where `i = x mod n`, `j = y mod n`

Error Diffusion (Floyd-Steinberg): Distributes quantization error to unprocessed neighbors → smooth gradients without banding.

Image Files

- Structure: **Header** (format, dimensions) + **Image Data**
- Compression: **Run-Length Encoding (RLE)** — e.g., `AAAAAABBBCC` → `(A,6),(B,3),(C,2)`

Exam Math

Q1a: What is quantization?

Quantization is the process of mapping a continuous range of values into a smaller, finite, discrete set. In image representation, it means converting continuous light intensities into discrete digital values — for example, mapping analog color intensity to an 8-bit value in the range 0–255.

Q1b: Maximum memory for 10 images using direct coding (640×520 pixels, 3 bytes/pixel)?

Given:

- Image dimensions: 640 × 520 pixels
- Color depth: 3 bytes per pixel (1 byte each for R, G, B)
- Number of images: 10

Step 1 — Total pixels per image:
$$640 \times 520 = 332{,}800 \text{ pixels}$$

Step 2 — Memory per image:
$$332{,}800 \text{ pixels} \times 3 \text{ bytes/pixel} = 998{,}400 \text{ bytes}$$

Step 3 — Total memory for 10 images:
$$998{,}400 \times 10 = \boxed{9{,}984{,}000 \text{ bytes}}$$

Converting: $9{,}984{,}000 \div 1024 = 9{,}750 \text{ KB} \approx 9.52 \text{ MB}$

Q1c: How many colors can be stored in a lookup table if each image's index memory must not exceed 500 KB?

Given:

- Image size: 640 × 520 = 332,800 pixels (from Q1b)
- Memory limit per image: 500 KB
- Goal: find maximum number of bits per pixel for the index, then find the number of colors

Step 1 — Convert 500 KB to bytes:
$$500 \times 1024 = 512{,}000 \text{ bytes}$$

Step 2 — Convert bytes to bits:
$$512{,}000 \times 8 = 4{,}096{,}000 \text{ bits}$$

Step 3 — Calculate available bits per pixel:
$$\frac{4{,}096{,}000 \text{ bits}}{332{,}800 \text{ pixels}} \approx 12.307 \text{ bits/pixel}$$

Step 4 — Round down to a whole number (must be integer bit depth):
$$\lfloor 12.307 \rfloor = 12 \text{ bits/pixel}$$

(We round down, not up, to stay within the 500 KB limit.)

Step 5 — Calculate number of representable colors:
$$2^{12} = \boxed{4{,}096 \text{ colors}}$$

Bonus Q: If 2-byte pixel values are used in a 24-bit lookup table, how many bytes does the lookup table occupy?

Given:

- Pixel size: 2 bytes = 16 bits → used as an index
- Each LUT entry: 24-bit color = 3 bytes

Step 1 — Number of LUT entries (one per possible index value):
$$2^{16} = 65{,}536 \text{ entries}$$

Step 2 — Size of each entry:
$$24 \text{ bits} \div 8 = 3 \text{ bytes per entry}$$

Step 3 — Total LUT size:
$$65{,}536 \times 3 = \boxed{196{,}608 \text{ bytes}}$$

Chapter 3: Scan Conversion

Scan-Converting a Point

$$X_{pixel} = \lfloor x + 0.5 \rfloor, \quad Y_{pixel} = \lfloor y + 0.5 \rfloor$$

Example: Map continuous point $(10.3,\ 20.7)$ to a pixel.

Step 1 — Add 0.5 to each coordinate:
$$x + 0.5 = 10.3 + 0.5 = 10.8$$
$$y + 0.5 = 20.7 + 0.5 = 21.2$$

Step 2 — Apply floor:
$$X_{pixel} = \lfloor 10.8 \rfloor = 10, \quad Y_{pixel} = \lfloor 21.2 \rfloor = 21$$

Result: Pixel plotted at $(10,\ 21)$

Line Drawing Algorithms

1. Slope-Intercept Method (y = mx + b)

Steps:

1. Compute slope: $m = \frac{y_2 - y_1}{x_2 - x_1}$
2. Compute y-intercept: $b = y_1 - m \cdot x_1$
3. For each integer x from x_1 to x_2: compute $y = mx + b$, round to nearest integer, plot $(x, \text{round}(y))$

Example — Line from (2, 3) to (6, 5):

Step 1 — Calculate slope:
$$m = \frac{5 - 3}{6 - 2} = \frac{2}{4} = 0.5$$

Step 2 — Calculate y-intercept:
$$b = y_1 - m \cdot x_1 = 3 - (0.5 \times 2) = 3 - 1 = 2$$

Step 3 — Iterate and compute y for each x (using $y = 0.5x + 2$):

x	Calculation	y (float)	$\text{round}(y)$	Plot
2	$0.5(2) + 2 = 1 + 2$	3.0	3	(2, 3)
3	$0.5(3) + 2 = 1.5 + 2$	3.5	4	(3, 4)
4	$0.5(4) + 2 = 2 + 2$	4.0	4	(4, 4)
5	$0.5(5) + 2 = 2.5 + 2$	4.5	5	(5, 5)
6	$0.5(6) + 2 = 3 + 2$	5.0	5	(6, 5)

✅ Advantage	❌ Disadvantage
Simple to understand	Floating-point multiplication every pixel
Direct calculation	Visual gaps when $m > 1$

---

2. DDA (Digital Differential Analyzer)

Steps:

1. Compute differences: $dx = x_2 - x_1$, $dy = y_2 - y_1$
2. Determine step count: $\text{step} = \max(|dx|, |dy|)$ — ensures no gaps regardless of slope
3. Compute per-step increments: $x_{inc} = dx / \text{step}$, $y_{inc} = dy / \text{step}$
4. Initialize: $x = x_1$, $y = y_1$
5. For each iteration: plot $(\text{round}(x), \text{round}(y))$, then $x = x + x_{inc}$, $y = y + y_{inc}$

Example — Line from (2, 3) to (6, 5):

Step 1 — Compute differences:
$$dx = 6 - 2 = 4, \quad dy = 5 - 3 = 2$$

Step 2 — Determine step count:
$$\text{step} = \max(|4|, |2|) = \max(4, 2) = 4$$

Step 3 — Compute increments:
$$x_{inc} = \frac{4}{4} = 1.0, \quad y_{inc} = \frac{2}{4} = 0.5$$

Step 4 — Initialize: $x = 2.0,\ y = 3.0$

Step 5 — Iterate (at each step: plot → add increments):

Iter	x	y	$\text{round}(x)$	$\text{round}(y)$	Plot	x_{next}	y_{next}
0	2.0	3.0	$2.0+1.0=3.0$	$3.0+0.5=3.5$	2	3	**(2,3)**
1	3.0	3.5	$3.0+1.0=4.0$	$3.5+0.5=4.0$	3	4	**(3,4)**
2	4.0	4.0	$4.0+1.0=5.0$	$4.0+0.5=4.5$	4	4	**(4,4)**
3	5.0	4.5	$5.0+1.0=6.0$	$4.5+0.5=5.0$	5	5	**(5,5)**
4	6.0	5.0	—	—	6	5	**(6,5)**

✅ Advantage	❌ Disadvantage
No multiplication	Still uses floating-point
No gaps in line	Cumulative rounding error on long lines

3. Bresenham's Line Algorithm (Integer Only)

Steps (for $m < 1$):

1. Compute: $dx = x_2 - x_1$, $dy = y_2 - y_1$
2. Pre-compute constants: $2dy$ and $2dy - 2dx$
3. Initialize: $x = x_1$, $y = y_1$, $P_0 = 2dy - dx$

4. At each step: plot (x, y), increment $x = x + 1$, then:
    - If $P_n < 0$: y stays, $P_{n+1} = P_n + 2dy$
    - If $P_n \geq 0$: $y = y + 1$, $P_{n+1} = P_n + 2dy - 2dx$
5. Repeat until $x = x_2$

Full Worked Example — Line from (2, 3) to (6, 5):

Step 1 — Compute differences:
$$dx = 6 - 2 = 4, \quad dy = 5 - 3 = 2$$

Step 2 — Pre-compute constants:
$$2dy = 2 \times 2 = 4$$
$$2dy - 2dx = 4 - 2(4) = 4 - 8 = -4$$

Step 3 — Initial decision variable:
$$P_0 = 2dy - dx = 4 - 4 = 0$$

Step 4 — Initialize: plot $(2, 3)$, then iterate:

Step	x	y	x_{next}	Action	y_{next}	p	p_{next}
0	2	3	3	y++	4	0	-4
1	3	4	4	y stays	4	-4	0
2	4	4	5	y++	5	0	-4
3	5	5	6	y stays	5	-4	—

Plotted pixels: (2,3), (3,4), (4,4), (5,5), (6,5)

Final pixels plotted: $(2,3),\ (3,4),\ (4,4),\ (5,5),\ (6,5)$

For $m \geq 1$ (steep lines):

- The roles of x and y swap compared to $m < 1$.
- Increment y each step, and decide whether to increment x based on P.
- Initial $P_0 = 2dx - dy$.
- If $P < 0$: x stays, $P = P + 2dx$.
- If $P \geq 0$: x increments, $P = P + 2dx - 2dy$.
  Memory tip: When slope flips, dx and dy swap in the formulas.

✅ Advantage	❌ Disadvantage
Integer-only (fastest)	More complex to implement for all slopes
No rounding needed	Different cases needed for $m>1$, negatives

---

Circle & Ellipse Drawing

8-Way Symmetry (Circles)

Compute one octant ($x \leq y$), mirror to get 8 points:

Octant	Transform	Pixel (from point (2,5))
1	(x, y)	(2, 5)
2	(y, x)	(5, 2)
3	$(-y, x)$	(−5, 2)
4	$(-x, y)$	(−2, 5)
5	$(-x, -y)$	(−2, −5)
6	$(-y, -x)$	(−5, −2)
7	$(y, -x)$	(5, −2)
8	$(x, -y)$	(2, −5)

> Ellipses use **4-way symmetry** only — x and y cannot be swapped.

Performance gain: 87.5% fewer calculations for circles; 75% for ellipses.

Bresenham's Circle Algorithm

Initialize: $x = 0$, $y = r$, $d_0 = 3 - 2r$

At each step (while $x \leq y$):

- Plot 8 symmetric points for current (x, y)
- If $d < 0$: y stays, $d = d + 4x + 6$
- If $d \geq 0$: $y = y - 1$, $d = d + 4x - 4y + 10$
- Always: $x = x + 1$

Full Worked Example — Circle with $r = 5$, centered at $(0,0)$:

Step 1 — Initialize:
$$x = 0, \quad y = 5$$
$$d_0 = 3 - 2(5) = 3 - 10 = -7$$

Step 2 — Iterate (note: d update uses x and y values *before* incrementing):

Step	x	y	x_{next}	Action	y_{next}	d	d_{next}
Init	0	5	1	d < 0	5	-7	-1
1	1	5	2	d < 0	5	-1	9
2	2	5	3	d >= 0	4	9	7
3	3	4	4	d >= 0	3	7	13

Stop: next $x = 5 > y = 3$, so first octant is complete.

Points computed (x, y): $(0,5), (1,5), (2,5), (3,4), (4,3)$ — each is mirrored using 8-way symmetry to produce the full circle.

Midpoint Circle Algorithm

Initialize: $x = 0$, $y = r$, $p_0 = 1 - r$

At each step (while $x \leq y$):

- Plot 8 symmetric points for current (x, y)
- If $p < 0$: y stays, $p = p + 2x + 3$
- If $p \geq 0$: $y = y - 1$, $p = p + 2x - 2y + 5$
- Always: $x = x + 1$

Full Worked Example — Circle with $r = 5$, centered at $(0,0)$:

Step 1 — Initialize:
$$x = 0, \quad y = 5$$
$$p_0 = 1 - r = 1 - 5 = -4$$

Step 2 — Iterate (note: p update uses x and y values *before* incrementing x, but *after* decrementing y when applicable):

Step	x	y	x_{next}	Action	y_{next}	p	d_{next}
Init	0	5	1	p < 0	5	-4	-1
1	1	5	2	p < 0	5	-1	4
2	2	5	3	p >= 0	4	4	3
3	3	4	4	p >= 0	3	3	6

Stop: next $x = 5 > y = 3$, first octant complete.

Points computed: $(0,5),\ (1,5),\ (2,5),\ (3,4),\ (4,3)$ — mirrored using 8-way symmetry.

> Midpoint is more adaptable than Bresenham's Circle — it is the standard method used for ellipses. For ellipses, split the curve into two regions at the point where slope $= -1$.

Scan-Converting Characters (Fonts)

Bitmap (Raster) Fonts

- Store characters as pre-calculated binary grids
- Rendering = copy grid to screen memory (BitBlt)

✅ Advantage	❌ Disadvantage
Extremely fast	Blocky when scaled

✅ Advantage	❌ Disadvantage
Pixel-perfect at native size	Separate file per size needed

Outline (Vector) Fonts (e.g., TrueType, OpenType)

- Store mathematical curves (Bézier) and lines
- Rendering pipeline: **Fetch geometry → Transform → Rasterize boundary → Scanline fill → Hint**

✅ Advantage	❌ Disadvantage
Infinitely scalable	Computationally expensive

| Single file for all sizes | Blurry at small sizes without hinting |

Scan-Converting Arcs, Sectors, Rectangles

Arcs

Three approaches:

1. **Trigonometric:** $x = h + r\cos\theta$, $y = k + r\sin\theta$ — accurate but slow (uses sin/cos)
2. **Polynomial:** $y = \sqrt{r^2 - x^2}$ — uses square roots, no symmetry
3. **Bresenham (avoid for arcs):** Must compute entire circle then check bounds — inefficient, risks infinite loop

Sectors (Pie Slice)

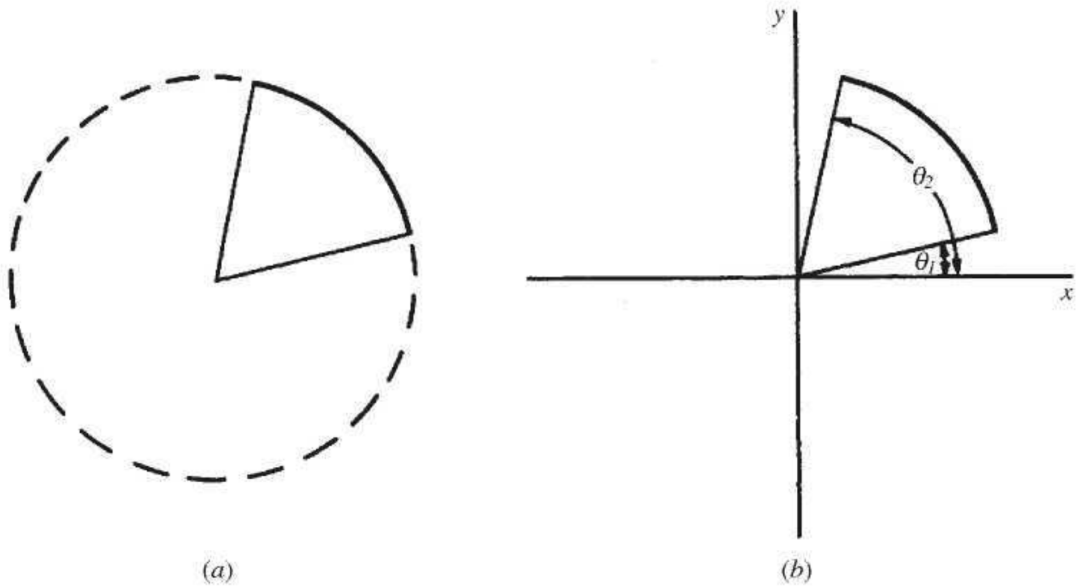

Fig. 3-13

Fig. 3-13: Scan-converting a sector (pie slice): (a) shows the arc and sector, (b) shows the angles and endpoints used for filling.

1. Draw arc from θ_1 to θ_2
2. Calculate endpoints: $(h + r\cos\theta_1,\ k + r\sin\theta_1)$ and $(h + r\cos\theta_2,\ k + r\sin\theta_2)$
3. Draw two lines from center to each endpoint (use Bresenham's line)

Rectangle (Axis-aligned)

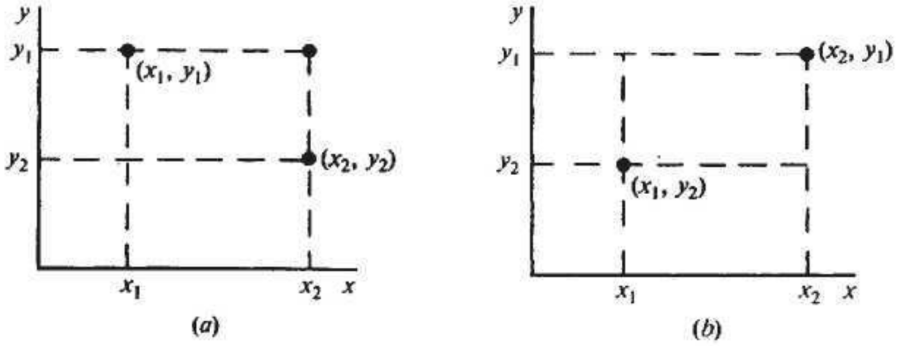

Fig. 3-16

Fig. 3-16: Scan-converting a rectangle: (a) and (b) show how the four corners are used to define the rectangle for filling or drawing edges.

- Input: two opposite corners (x_1, y_1) and (x_2, y_2)
- Derive: (x_1, y_2) and (x_2, y_1)
- Draw 4 edges using any line algorithm

Region Filling

Interior-defined region: A region where filling begins from a seed point inside the boundary and continues outward until the boundary is reached.

Boundary-defined region: A region enclosed by a specific boundary color, where filling begins from a seed point inside the area and continues outward pixel-by-pixel until the specific boundary color is reached.

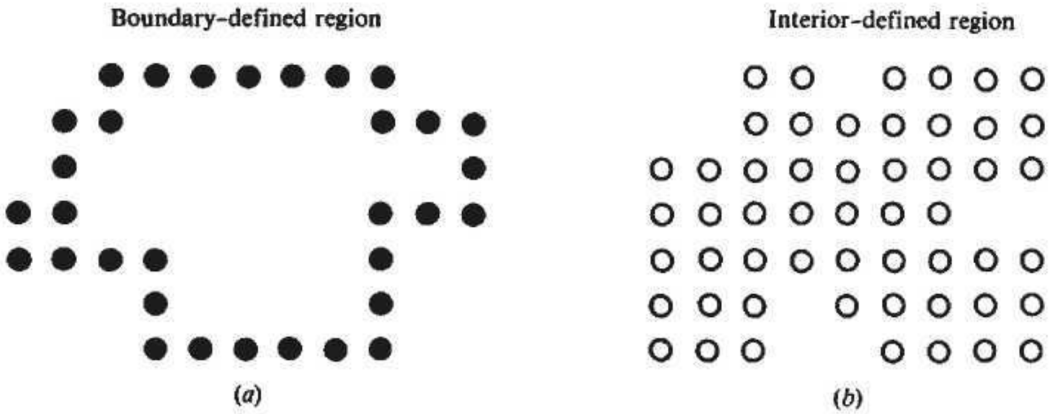

Fig. 3-17

Fig. 3-17: (a) Boundary-defined region: filling is constrained by a boundary color. (b) Interior-defined region: filling spreads outward from a seed point until the boundary is reached.

Pixel Connectivity

Type	Neighbors
4-Connected	Up, Down, Left, Right only
8-Connected	All 8 surrounding pixels (includes diagonals)

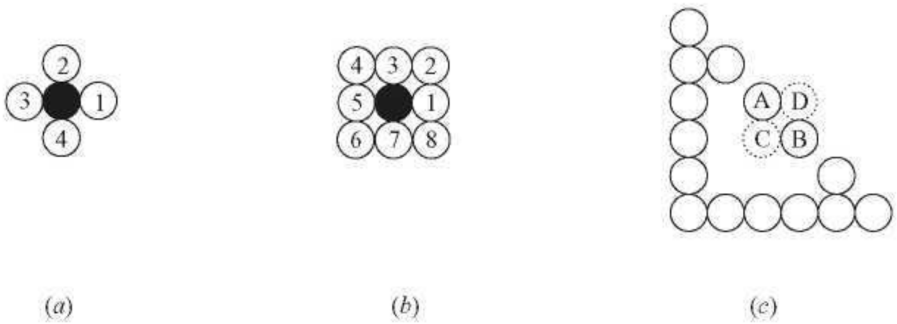

Fig. 3-18 4-connected vs. 8-connected pixels.

Fig. 3-18: (a) 4-connected neighbors: only up, down, left, right. (b) 8-connected neighbors: includes diagonals. (c) Example showing how connectivity affects region filling.

> **Critical rule:** If boundary is 8-connected → fill must be 4-connected, and vice versa (prevents leaking).

---

1. Boundary-Fill (Recursive)

Used when region is enclosed by a specific **boundary color**.

Steps:

1. Start at seed pixel
2. If pixel = fill color OR boundary color → stop
3. Else: paint pixel with fill color, recurse on neighbors

✅ Advantage	❌ Disadvantage
Works on irregular shapes	Stack overflow on large regions
Easy to implement	Fails if boundary is multi-colored

---

2. Flood-Fill (Recursive)

Used when region is defined by a uniform **interior color** (e.g., paint bucket tool).

Steps:

1. Start at seed, record `Old_Color`
2. If pixel ≠ `Old_Color` OR already = `New_Color` → stop
3. Else: paint with `New_Color`, recurse on neighbors

✅ Advantage	❌ Disadvantage
No boundary color needed	Stack overflow risk
Natural stopping condition	Cannot fill gradients

---

3. Scanline Fill (Optimized Polygon Fill)

We represent a polygonal region in terms of a sequence of vertices $V_1, V_2, V_3, \ldots$, that are connected by edges $E_1, E_2, E_3, \ldots$, (see Fig. 3-19). We assume that each vertex V_i has already been scan-converted to integer coordinates (x_i, y_i).

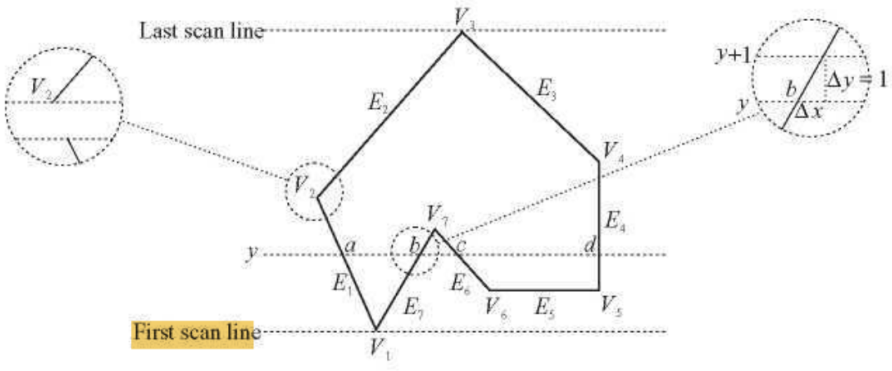

Fig. 3-19 Scan-converting a polygonal region.

Fig. 3-19: Scan-converting a polygonal region.

Table 3-1 An edge list.

Edge	y_{min}	y_{max}	x coordinate of vertex with $y = y_{min}$	$1/m$
E_1	y_1	$y_2 - 1$	x_1	$1/m_1$
E_7	y_1	y_7	x_1	$1/m_7$
E_4	y_5	$y_4 - 1$	x_5	$1/m_4$
E_6	y_6	y_7	x_6	$1/m_6$
E_2	y_2	y_3	x_2	$1/m_2$
E_3	y_4	y_3	x_4	$1/m_3$

Table 3-1: An edge list for scanline fill.

Fills row-by-row using the polygon's geometric edge data. Eliminates recursion.

Core Concepts:

Concept	Explanation
Sweeping Scanline	Process one horizontal line at a time, bottom to top
Vertex Problem	Fix: shorten lower edge by 1 ($y_{max} - 1$) at monotonic vertices
Horizontal Rule	Ignore horizontal edges
Edge Activation Rule	Edge is active from y_{min} up to (but not including) y_{max}; it is deactivated/removed after the scanline at y_{max} is processed.
Odd-Even Rule	Fill pixels between paired intersection points
Edge Coherence	$x_{new} = x_{old} + \frac{1}{m}$ — no recalculation per row

Algorithm Steps:

1. **Build Edge Table (ET):** For each non-horizontal edge compute $y_{min}, y_{max}, x, \frac{1}{m}$; apply $y_{max}-1$ rule; sort by y_{min}
2. **Initialize:** $y = \min(y_{min})$
3. **Update AEL:** Move edges with $y_{min} = y$ into Active Edge List
4. **Sort AEL** by x (left to right)
5. **Fill:** Pair edges (0&1, 2&3, …); fill between each pair
6. **Remove** edges where $y_{max} = y$
7. **Increment:** $y = y + 1$; update $x_{new} = x + \frac{1}{m}$ for remaining AEL edges

8. **Repeat** until ET and AEL are empty

Worked Example — Q2(b): Vertices $(-4,0),(0,2),(4,0),(2,4),(6,4),(0,8),(-6,4),(-4,4)$

Step 1 — Form edges and discard horizontal ones:

Edge	From → To	Horizontal?
E1	$(-4,0)\to(0,2)$	No — keep
E2	$(0,2)\to(4,0)$	No — keep
E3	$(4,0)\to(2,4)$	No — keep
E4	$(2,4)\to(6,4)$	**Yes — discard**
E5	$(6,4)\to(0,8)$	No — keep
E6	$(0,8)\to(-6,4)$	No — keep
E7	$(-6,4)\to(-4,4)$	**Yes — discard**
E8	$(-4,4)\to(-4,0)$	No — keep

Step 2 — Compute ET values using $\frac{1}{m} = \frac{x_2 - x_1}{y_2 - y_1}$:

- **E1** $(-4,0)\to(0,2)$: $y_{min}=0,\ y_{max}=2,\ x=-4,\ \frac{1}{m}=\frac{0-(-4)}{2-0}=\frac{4}{2}=2$
- **E2** $(0,2)\to(4,0)$: $y_{min}=0,\ y_{max}=2,\ x=4,\ \frac{1}{m}=\frac{4-0}{0-2}=\frac{4}{-2}=-2$
- **E3** $(4,0)\to(2,4)$: $y_{min}=0,\ y_{max}=4,\ x=4,\ \frac{1}{m}=\frac{2-4}{4-0}=\frac{-2}{4}=-0.5$
- **E5** $(6,4)\to(0,8)$: $y_{min}=4,\ y_{max}=8,\ x=6,\ \frac{1}{m}=\frac{0-6}{8-4}=\frac{-6}{4}=-1.5$
- **E6** $(0,8)\to(-6,4)$: $y_{min}=4,\ y_{max}=8,\ x=-6,\ \frac{1}{m}=\frac{-6-0}{4-8}=\frac{-6}{-4}=1.5$
- **E8** $(-4,4)\to(-4,0)$: $y_{min}=0,\ y_{max}=4,\ x=-4,\ \frac{1}{m}=\frac{-4-(-4)}{0-4}=\frac{0}{-4}=0$

Step 3 — Final Edge Table (sorted by y_{min}):

Edge	y_{min}	y_{max}	x at y_{min}	$1/m$
E1	0	2	−4	2
E2	0	2	4	−2
E3	0	4	4	−0.5
E8	0	4	−4	0
E5	4	8	6	−1.5
E6	4	8	−6	1.5

Step 4 — Initialize: $y = 0$ (smallest y_{min})

Scanline $y = 0$:

- AEL: add E1, E2, E3, E8 (all have $y_{min} = 0$)
- Sort AEL by x: $-4\ (\text{E8}),\ -4\ (\text{E1}),\ 4\ (\text{E3}),\ 4\ (\text{E2})$
- Pair (E8,E1) → fill from -4 to -4 → zero width, **no fill**
- Pair (E3,E2) → fill from 4 to 4 → zero width, **no fill**
- No edges have $y_{max} = 0$, so none are removed
- Update x values: $x_{new} = x_{old} + \frac{1}{m}$

Edge	Old x	$1/m$	New x
E1	–4	2	$-4 + 2 = -2$
E2	4	–2	$4 + (-2) = 2$
E3	4	–0.5	$4 + (-0.5) = 3.5$
E8	–4	0	$-4 + 0 = -4$

Scanline $y = 1$:

- No new edges enter AEL
- AEL sorted by updated x: $-4\ (\text{E8}),\ -2\ (\text{E1}),\ 2\ (\text{E2}),\ 3.5\ (\text{E3})$
- Pair (E8, E1) → **fill from $x = -4$ to $x = -2$ ✔**
- Pair (E2, E3) → **fill from $x = 2$ to $x = 3.5$ ✔**
- Update x values:

Edge	Old x	$1/m$	New x
E1	–2	2	$-2 + 2 = 0$
E2	2	–2	$2 + (-2) = 0$
E3	3.5	–0.5	$3.5 - 0.5 = 3$
E8	–4	0	$-4 + 0 = -4$

Scanline $y = 2$:

- Remove E1 and E2 ($y_{max} = 2 = y$)
- AEL now: E3, E8 (sorted: $-4,\ 3$)
- **Fill from $x = -4$ to $x = 3$ ✔**
- Update:

Edge	Old x	$1/m$	New x
E3	3	–0.5	$3 - 0.5 = 2.5$
E8	–4	0	-4

Scanline $y = 3$:

- AEL: E3, E8 (sorted: $-4,\ 2.5$)

- **Fill from $x = -4$ to $x = 2.5$ ✔**
- Update: E3 → $2.5 - 0.5 = 2$; E8 → -4

Scanline $y = 4$:

- Remove E3 and E8 ($y_{max} = 4 = y$)
- Add E5 and E6 from ET ($y_{min} = 4$)
- AEL: E5 $(x=6)$, E6 $(x=-6)$ → sorted: $-6,\ 6$
- **Fill from $x = -6$ to $x = 6$ ✔**
- Update: E5 → $6 - 1.5 = 4.5$; E6 → $-6 + 1.5 = -4.5$

(Continue similarly for $y = 5, 6, 7$ until $y = 8$ where E5 and E6 are removed and both ET and AEL are empty.)

✅ Advantage	❌ Disadvantage
No recursion	ET and AEL maintenance overhead
Fast via edge coherence	AEL re-sort needed when edges cross
Handles concave polygons & holes	

Common Mistakes:

- Including horizontal edges in ET
- Forgetting to sort AEL by x each scanline
- Calculating $\frac{1}{m}$ as $\frac{\Delta y}{\Delta x}$ instead of $\frac{\Delta x}{\Delta y}$
- Forgetting $y_{max} - 1$ rule at monotonic vertices

Anti-Aliasing

Aliasing Artifacts

- Visual distortions caused by approximating continuous objects with discrete pixels.

Artifact	Description
Staircase (Jaggies)	Jagged edges on diagonal lines/curves
Unequal Brightness	Diagonals appear dimmer than horizontal/vertical lines. Because distance between two diagonal pixel is larger
Picket Fence Problem	When an object does not fit into, the pixel grid properly
Edge Flickering	Edge vibrate during animation
Moire Pattern	Interferance occur when high frequency texture are sampled

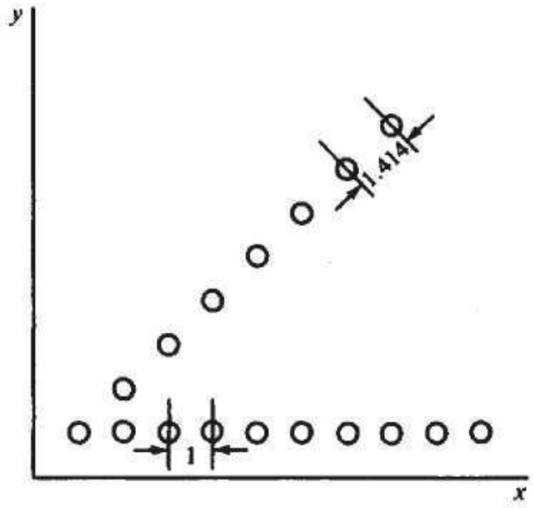

Fig. 3-23

Fig.: Unequal Brightness

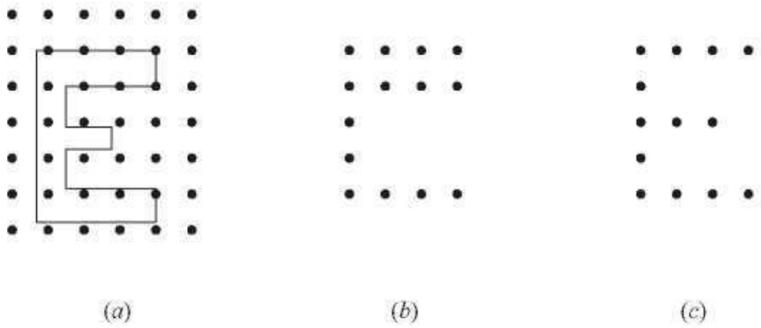

Fig. 3-25 Scan-converting an outline font.

Fig.: Picket Fence Problem with the outline font.
Suppose we want to scan-convert the uppercase character "E" in (a) from its outline description to a bitmap consisting of pixels inside the region defined by the outline. The result in (b) exhibits both asymmetry (the upper arm of the character is twice as thick as the other parts) and dropout (the middle arm is absent). A slight adjustment and/or realignment of the outline can lead to a reasonable outcome (c).

. Why Anti-Aliasing is Needed

- Simply increasing resolution reduces jaggedness but **requires a lot of memory and processing**.
- Anti-aliasing **smooths edges without increasing resolution**, making images appear natural.

. Main Anti-Aliasing Techniques with Examples

A. Pre-Filtering

- Works **before converting to pixels**.
- Each pixel's color is set **based on how much of it is covered by the object**.

Example:

- Pixel partially covered by a white line:

  Coverage = 50% (0.5)
  Object color = White (1)
  Background color = Black (0)
  Pixel color = 1 × 0.5 + 0 × 0.5 = 0.5 → gray

- Fully inside pixel → full white, outside → black.

Result: Smooth edge; shape remains sharp in the center.

B. Supersampling (Post-Filtering)

- Each pixel is divided into **subpixels** (e.g., 3×3).
- Count subpixels inside the object → average their colors → final pixel color.

Example:

- Pixel divided into 3×3 = 9 subpixels
- 6 subpixels are inside a line → coverage = 6/9 ≈ 0.67
- Object color = white (1), background = black (0)
- Pixel color = 1 × 0.67 + 0 × 0.33 = 0.67 → gray

Visual Idea:

[• • •]
[• o o]
[o o o]

- • = subpixel inside object
- o = subpixel outside object
- Coverage = 6/9 → pixel color 0.67 (blended)

Result: Smooth diagonal lines and edges.

C. Pixel Phasing

- Hardware-based: shifts pixels **slightly from their normal positions** to align with true line/contour.
- Reduces stair-step effect without blurring edges.

Example:

- A diagonal line across a pixel grid may look jagged.

- Pixel positions are shifted **fractionally toward the line**, so the line looks smoother.

Visual Idea (simplified):

Before: After:
• o • •
o • o •

- Line edges appear aligned → stair-step effect minimized.

D. Gray-Level / Color Blending

- Uses **intermediate shades or colors** between object and background.
- Only edge pixels are blended, making transitions smooth.

Example:

- White line on black background
- Edge pixel = 50% covered → gray
- Edge pixel = 25% covered → dark gray

Visual Idea:

```
Background → Dark Gray → Gray → White (center of line)
```

Result: Human eye sees a smooth, continuous line.

In short:
Anti-aliasing tricks your eyes by **blending colors, averaging sub-pixels, or shifting pixels slightly**, so edges appear **smooth and natural**, even on low-resolution screens.

Chapter 4: Two-Dimensional Transformations

Introduction
Transformation is the mathematical process of simulating the spatial manipulation of objects. There are two primary viewpoints for transformations:

1. **Geometric Transformation:** The coordinate system remains stationary, and the object itself is moved or altered.
2. **Coordinate Transformation:** The object remains stationary, and the coordinate system is moved or altered around it.

4.1 Geometric Transformations
This section defines the fundamental operations for manipulating a point $P(x,y)$ to a new position $P'(x',y')$:

- **Translation (T_v):** Displacing an object by a specific distance and direction using a vector.

- $x' = x + t_x$ and $y' = y + t_y$

displacement is given by the vector $\mathbf{v} = t_x\mathbf{I} + t_y\mathbf{J}$, the new object point $P'(x', y')$ can be found by applying the transformation $T_{\mathbf{v}}$ to $P(x, y)$ (see Fig. 4-2).

$$P' = T_{\mathbf{v}}(P)$$

where $x' = x + t_x$ and $y' = y + t_y$.

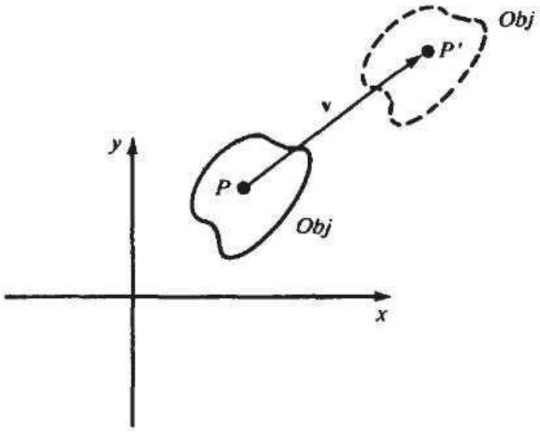

Fig. 4-2

Fig. 4-2: Translation of an object by vector $\vec{v}$, moving point P to P'.

- **Rotation (R_θ):** Rotating an object θ° about the origin. Counterclockwise is positive, and clockwise is negative.
  - $x' = x\cos\theta - y\sin\theta$ and $y' = x\sin\theta + y\cos\theta$

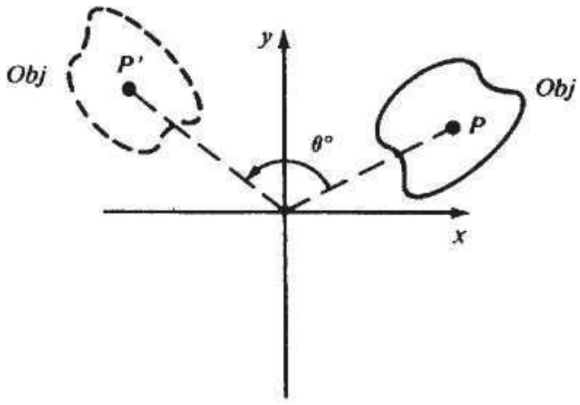

Fig. 4-3

Fig. 4-3: Rotation of an object by θ degrees, moving point P to P'.

- **Scaling (S_{s_x, s_y}):** Expanding or compressing an object using scaling constants s_x and s_y. Only the origin remains fixed. Uniform scaling occurs when $s_x = s_y$.
  - $x' = s_x x$ and $y' = s_y y$

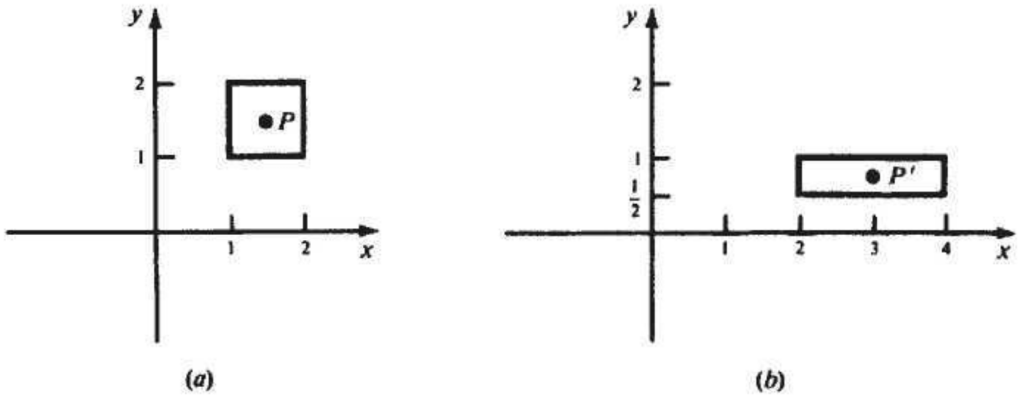

Fig. 4-4

Fig. 4-4: scaling transformation with scaling factors $s_x = 2$ and $s_y = 1/2$.

- **Mirror Reflection (M):** Creating a mirrored image of an object across an axis (e.g., about the x-axis: $x' = x$ and $y' = -y$).

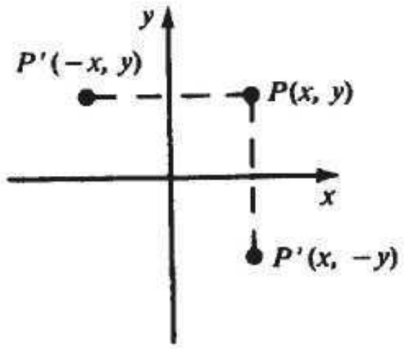

Fig. 4-5

Fig. 4-5: Mirror reflection of a point $P(x, y)$ across the x-axis and y-axis, resulting in $P'(x, -y)$ and $P'(-x, y)$.

- **Inverse Transformations:** Every geometric transformation has a direct inverse that reverses the operation (e.g., the inverse of translating by v is translating by $-v$).
    - **Translation:** T_{-v} is the inverse of T_v (translation by $-v$ reverses translation by v).
    - **Rotation:** $R_{-\theta}$ is the inverse of R_{θ} (rotation by $-\theta$ undoes rotation by θ).
    - **Scaling:** $S_{1/s_x, 1/s_y}$ is the inverse of S_{s_x, s_y} (scaling by the reciprocal factors undoes the original scaling).
    - **Mirror Reflection:** The inverse of a mirror reflection is itself (applying the same reflection twice returns the object to its original position).

4.2 Coordinate Transformations

Instead of modifying the object's points, a new coordinate system ($x'y'$) is defined relative to the old one (xy). Transformations (translation, rotation, scaling, and reflection) are applied to the axes, which inversely affects the coordinate descriptions of the stationary points.

4.3 Composite Transformations & Homogeneous Coordinates

- **Composition:** Complex transformations are built by stringing together basic transformations. For example, to magnify an object while keeping its center $C(h,k)$ fixed, you must: (1) Translate the object so C is at the origin, (2) Scale it, and (3) Translate it back to its original position.
- **Matrix Representation:** Rotation, scaling, and reflection can be represented as 2×2 matrices, but translation cannot.
- **Homogeneous Coordinates:** To allow all transformations (including translation) to be treated as matrix multiplications, points are represented as 3D vectors $[x, y, 1]^T$, and transformations are upgraded to **3×3 matrices**.
- **Composite Transformation Matrix (CTM):** Because all basic transformations are now 3×3 matrices, they can be multiplied (concatenated) together into a single CTM, which is highly computationally efficient.

4.4 Instance Transformations

- **Instances:** A complex picture often uses the same object multiple times (e.g., a wheel on a car). An object is defined once in its own local coordinate system, and an **instance transformation** converts those local coordinates into the master "picture" coordinates. This typically involves scaling, rotating, and translating the object into its proper place.
- **Multilevel/Nested Structures:** Objects can be made of sub-objects (e.g., apples on a branch, branch on a tree). To render these efficiently, instance transformations are concatenated across levels so that the lowest-level object is transformed directly into the final picture coordinates in one step.